

IELTS Listening Spelling Trainer

Build specification for Claude Code — a browser-based dictation simulator for letter-by-letter spelling of surnames, postcodes and alphanumeric codes

Field	Value
Version	1.0 (prototype)
Date	21 September 2026
Author	Mikhail Maidan (Maggot)
Purpose	Hand this document to Claude Code as the complete brief for building the first working prototype.
Stack	Next.js (App Router) + React + TypeScript + Tailwind CSS; Web Speech API; no database

How to use this document with Claude Code. Paste or attach it and say: *“Build the application described in this specification. Follow the iteration plan in Section 12 and stop after each iteration for review.”* Every section is written to be unambiguous enough to implement without further clarification; where a decision is intentionally left open, it is marked **[OPEN]**.

Contents

1. Problem statement and goal	3
1.1 Success criteria for the prototype	3
1.2 Explicitly out of scope for v1	3
2. Technology stack	3
3. Project structure	4
4. Core data model	4
5. Content generation (on the fly, no lists)	5
5.1 Surnames — syllable-pattern generator	5
5.2 UK postcodes — from format rules	6
5.3 US ZIP codes	6
5.4 Free-form alphanumeric / reference codes	6
5.5 Mixed mode	6
5.6 Preset definitions (length and character mix)	6
6. Speech: tokenisation, grouping and pronunciation	7
6.1 Tokeniser — string → SpeechToken[]	7
6.2 Pronunciation map — never pass a bare letter to the synthesiser	7
6.3 Speaker — timed utterance queue	8
6.4 Voices — enumeration and picker	8
7. Speed control and adaptive timing	9
7.1 Manual control	9
7.2 Adaptive mode (default on)	9
8. Training loop and keyboard flow	9
8.1 Input behaviour	9
8.2 Live feedback while dictating (optional, toggle in settings)	10
9. Scoring and results	10
9.1 Character-level diff	10
9.2 Result view content	10
9.3 Session statistics (stats bar + session summary on demand, key T)	10
10. Settings panel (side drawer, key S)	10
11. UI layout (deliberately minimal)	11
12. Iteration plan for Claude Code	12
13. Known risks and mitigations	12
14. Future extensions (not for v1)	12

1. Problem statement and goal

In the IELTS Listening test, candidates must transcribe names, surnames, postcodes, reference numbers and similar strings that the speaker spells out letter by letter (e.g.

“W-O-J-C-I-E-C-H-O-W-S-K-I”, “the postcode is S-W-one-A, two-A-A”). Speakers routinely use grouped forms such as “**double L**”, “**triple seven**”, and read **0** as “oh” or “zero”. The difficulty is not vocabulary but *real-time keystroke tracking*: hearing a character and typing it immediately, without falling behind.

The goal is a small web application that acts as a personal trainer for exactly this skill. The machine dictates a string character by character using the browser's built-in speech synthesis; the user types along in real time; the app scores the attempt, shows a character-level diff, and adapts the dictation speed to the user's actual performance.

1.1 Success criteria for the prototype

- The user can start a session with two keystrokes or fewer and never needs the mouse during practice.
- Every dictated item is generated on the fly — **no bundled lists of surnames or postcodes, no databases**.
- The gap between dictated characters is user-adjustable to a tenth of a second and can adapt automatically.
- Voices/accents are chosen from whatever the browser exposes at runtime (British, American, Australian, ...).
- “double X” / “triple X” grouping is supported for letters and digits with three modes (never / always / random).
- Typing latency is imperceptible: the input reflects each keystroke immediately, even while audio is playing.
- All settings and session statistics persist across page reloads via *localStorage*.

1.2 Explicitly out of scope for v1

- Server-side storage, accounts, login.
- Pre-recorded audio or cloud TTS services (paid or free).
- Elaborate visual design. Clean Tailwind defaults are sufficient; polish comes after the app has been used.
- Full IELTS listening passages, comprehension questions, or anything beyond spelling dictation.
- Mobile-first layout. Desktop keyboard use is the target; it should merely not break on a tablet.

2. Technology stack

Layer	Choice	Rationale
Framework	Next.js 14+ (App Router), TypeScript	One project for the client app plus optional API routes later. Static export is fine for v1 (output: 'export').
UI	React 18 + Tailwind CSS	Minimal styling effort; utility classes keep the prototype fast to iterate.
Speech	Web Speech API — <code>window.speechSynthesis</code>	Free, built into the browser, no key, no quota. Voices come from the OS and (in Chrome) Google network voices.

Layer	Choice	Rationale
State	React state + a small useReducer store; zustand is acceptable if preferred	The app is small; avoid heavy state libraries.
Persistence	localStorage (settings, adaptive speed, session history)	No backend needed. Wrap in try/catch.
Generation	Pure TypeScript functions in /lib/generators	Deterministic given a seeded RNG so a failed item can be replayed exactly.
Testing	Vitest for generator and grouping logic	Generators and the “double/triple” tokenizer are the highest-risk pure logic; unit-test them.
Target browser	Chrome / Chromium desktop (primary); Edge, Firefox, Safari secondary	Chrome has the widest voice set and the most reliable speechSynthesis.

No backend in v1. Everything is generated client-side. Next.js API routes may be added in a later iteration (e.g. to fetch real surname frequency data) but must not be required for the prototype to work.

3. Project structure

```

app/
  layout.tsx      root layout, loads fonts, wraps in SettingsProvider
  page.tsx        the single training screen (all three views are state-driven)
components/
  TrainerInput.tsx the large monospace input + live character display
  SettingsPanel.tsx difficulty, speed, voice, grouping, content-type controls
  ResultsView.tsx  per-item diff + session summary
  VoicePicker.tsx  runtime voice enumeration, grouped by accent
  StatsBar.tsx     accuracy, current gap (ms), streak
lib/
  generators/
    surname.ts      syllable-pattern surname generator (British / American flavours)
    ukPostcode.ts   UK postcode generator from format rules
    usZip.ts        5-digit and ZIP+4 generator
    alnum.ts        free-form alphanumeric / reference-code generator
    index.ts        generate(contentType, preset, rng) -> Item
  speech/
    tokenizer.ts    string -> SpeechToken[] (handles double/triple grouping)
    pronounce.ts    character -> spoken form mapping table
    speaker.ts      queue of SpeechSynthesisUtterance with timed gaps
    voices.ts       enumerate + filter voices, handle 'voiceschanged'
  scoring/
    diff.ts         character-level diff (expected vs typed)
    adaptive.ts     gap adjustment rules
    rng.ts          seeded PRNG (mulberry32 or similar)
    storage.ts      typed localStorage wrapper
    types.ts        shared types (Settings, Item, Attempt, Session)

```

4. Core data model

```

type ContentType = 'surname' | 'ukPostcode' | 'usZip' | 'alnum' | 'mixed';
type Preset = 'easy' | 'medium' | 'hard';
type GroupingMode = 'never' | 'always' | 'random';
type ZeroStyle = 'oh' | 'zero' | 'random';

interface Settings {
  contentType: ContentType;
  preset: Preset;
  gapMs: number;           // pause between characters, 200..2000, step 50
  adaptive: boolean;       // auto-adjust gapMs after each item
}

```

```

grouping: GroupingMode;      // "double L" / "triple 7"
zeroStyle: ZeroStyle;
voiceURI: string | null;     // SpeechSynthesisVoice.voiceURI, null = default for locale
locale: 'en-GB' | 'en-US' | 'en-AU' | 'en-IE' | 'en-IN' | 'en-ZA' | 'any';
rate: number;                // SpeechSynthesisUtterance.rate, 0.7..1.3
announceType: boolean;       // say "surname:" / "postcode:" before spelling
readWholeFirst: boolean;     // say the whole word once before spelling (surnames only)
allowReplay: boolean;        // R key replays the item once, counted in stats
caseSensitive: false;        // always false in v1; kept for future
ignoreSpaces: boolean;       // postcode space optional in scoring
}

interface SpeechToken {
  text: string;               // what is spoken, e.g. "double L", "ay", "seven"
  chars: string;              // the characters it represents, e.g. "LL", "A", "7"
}

interface Item {
  id: string;                  // uuid
  seed: number;                // rng seed so the item can be regenerated exactly
  contentType: ContentType;
  target: string;              // canonical string, e.g. "SW1A 2AA"
  tokens: SpeechToken[];       // what will actually be spoken, in order
  createdAt: number;
}

interface Attempt {
  itemId: string;
  typed: string;
  correct: boolean;
  charAccuracy: number;        // 0..1, from diff
  gapMs: number;               // gap used for this item
  replays: number;
  keystrokeTimes: number[];    // ms offset of each keystroke from dictation start
  finishedAt: number;
}

interface Session {
  id: string;
  startedAt: number;
  attempts: Attempt[];
  settingsSnapshot: Settings;
}

```

5. Content generation (on the fly, no lists)

All content is produced by pure functions that take a seeded RNG. This guarantees unlimited variety, no bundled data, and exact reproducibility of any item from its seed (needed for “retry this one”).

5.1 Surnames — syllable-pattern generator

Surnames are assembled from onset / nucleus / coda fragments so that the output *looks and sounds like* a plausible British or American surname without being drawn from any list. Two flavour tables are defined inline in `surname.ts`:

- **Anglo/British flavour** — onsets: B, Br, Ch, Cl, D, F, Fl, G, Gr, H, J, K, L, M, N, P, Pr, R, S, Sh, St, Str, T, Th, Tr, W, Wh, Wr; nuclei: a, e, i, o, u, ea, ee, ai, oo, ou, ei, au; codas: ck, ll, tt, rd, rn, nd, ng, th, gh, ght, ss, mp, ft, ld, rk, ly, ey, ton, son, ley, worth, bury, field, ford, wood, well, ham, shaw.
- **American/immigrant flavour** (adds harder clusters typical of Polish, German, Italian, Dutch origin surnames) — onsets: Sz, Cz, Wo, Kr, Schw, Zh, Dz, Prz, Ng, Vl, Mc, O'; codas: ski, cki, wicz, czyk, mann, berg, stein, witz, elli, ucci, ini, ough, aux, dt.

- The generator picks 1–4 syllables depending on preset, capitalises the first letter, and optionally inserts an apostrophe or hyphen (hard preset only, e.g. O'Kearney-Sz).
- Post-filters: reject strings longer than the preset maximum, reject four identical consecutive letters, reject anything containing an obviously unpronounceable run of 4+ consonants unless the run is in the coda table.
- **[OPEN]** A later iteration may add an API route that pulls a real-name frequency source; v1 must not depend on it.

5.2 UK postcodes — from format rules

Generate from the six official patterns rather than from any list. Letters exclude those never used in the corresponding position (outward first letter never Q, V, X; inward letters never C, I, K, M, O, V):

Pattern	Example	Preset
A9 9AA	M1 1AE	easy
A99 9AA	B33 8TH	easy
AA9 9AA	CR2 6XH	medium
AA99 9AA	DN55 1PT	medium
A9A 9AA	W1A 0AX	hard
AA9A 9AA	EC1A 1BB	hard

Postcodes are spoken with the outward and inward halves separated by a slightly longer pause (1.5 × gapMs) and the word “space” is *not* spoken by default (IELTS speakers don't say it). The letter **O** and the digit **0** are both possible; the speaker says “oh” or “zero” for the digit according to zeroStyle.

5.3 US ZIP codes

- Easy: 5 digits. Medium: 5 digits with at least one repeated digit (to trigger “double”). Hard: ZIP+4 (90210-1234), hyphen spoken as “dash” or “hyphen” at random.

5.4 Free-form alphanumeric / reference codes

Booking references, membership numbers, flight numbers, licence plates. Generated from a template string per preset, where L = letter, D = digit, - = literal hyphen, / = literal slash:

Preset	Templates (one chosen at random)	Notes
easy	LLDD, DDDL, LLL-DD	4–6 chars, no confusable pairs
medium	LLDDDL, DLLDDD, LL-DDDD, DDLDD	6–8 chars, may include B/P, M/N, S/F, I/E confusables
hard	LLDDLDD, DDD/LLL-DD, LLLDDDL, LDLDLDD	8–12 chars, repeated runs likely, mixed separators

5.5 Mixed mode

Each item's content type is drawn at random from the types enabled in settings, weighted equally. This is the recommended default once the user is comfortable with each type individually.

5.6 Preset definitions (length and character mix)

Preset	Surname	UK postcode	US ZIP	Alnum	Confusables
easy	4–6 letters, 1–2 syllables, Anglo flavour	A9 9AA, A99 9AA	5 digits	4–6 chars	avoided
medium	6–9 letters, 2–3 syllables, both flavours	AA9 9AA, AA99 9AA	5 digits, ≥1 repeat	6–8 chars	allowed
hard	9–14 letters, 3–4 syllables, immigrant flavour, hyphen/apostrophe possible	A9A 9AA, AA9A 9AA	ZIP+4	8–12 chars	encouraged

Speed is not part of a preset. The pause between characters (gapMs) is a fully independent control so that hard content can be practised at a slow pace and vice versa.

6. Speech: tokenisation, grouping and pronunciation

6.1 Tokeniser — string → SpeechToken[]

Before anything is spoken, the target string is scanned for runs of identical characters and converted to tokens. Rules:

- A run of exactly 2 identical characters → one token “double X” if grouping is **always**; two single tokens if **never**; a coin flip (50/50) if **random**.
- A run of exactly 3 → “triple X” under the same rule. Runs of 4+ are prevented by the generators.
- Grouping applies to **letters and digits** alike (“double L”, “double seven”).
- Spaces produce no token but add an extra pause (1.5 × gapMs). Hyphen → “dash” or “hyphen” (random); apostrophe → “apostrophe”; slash → “slash”.
- When readWholeFirst is on and the item is a surname, a leading token with the whole word is emitted, followed by a 2 × gapMs pause, then “that's” (optional), then the spelling.
- Each token records the characters it stands for (chars) so the live-feedback display can highlight which token the user is currently expected to be typing.

```
tokenize("SW1A 0AA", { grouping: 'always', zeroStyle: 'oh' }) ->
[ {text:"ess",chars:"S"}, {text:"double you",chars:"W"}, {text:"one",chars:"1"},
  {text:"ay",chars:"A"}, {text:"<pause 1.5x>",chars:" "}, {text:"oh",chars:"0"},
  {text:"double ay",chars:"AA"} ]
```

6.2 Pronunciation map — never pass a bare letter to the synthesiser

Speech engines often read single letters badly (“A” becomes the article *uh*, short letters get clipped). Every character is therefore mapped to a spelled-out spoken form. The table is the single source of truth in pronounce.ts; the British/American column is selected from the chosen voice's language tag.

Char	Spoken (en-GB)	Spoken (en-US)	Char	Spoken (en-GB)	Spoken (en-US)
A	ay	ay	N	en	en
B	bee	bee	O	oh	oh
C	see	see	P	pee	pee
D	dee	dee	Q	cue	cue
E	ee	ee	R	ar	ar

Char	Spoken (en-GB)	Spoken (en-US)	Char	Spoken (en-GB)	Spoken (en-US)
F	eff	eff	S	ess	ess
G	jee	jee	T	tee	tee
H	aitch	aitch	U	you	you
I	eye	eye	V	vee	vee
J	jay	jay	W	double you	double you
K	kay	kay	X	ex	ex
L	el	el	Y	why	why
M	em	em	Z	zed	zee
0	oh / zero	zero / oh	-	dash / hyphen	dash / hyphen
1–9	one ... nine	one ... nine	' /	apostrophe, slash	apostrophe, slash

Implementation note. Test each spoken form against the actual Chrome voices during iteration 2 and adjust spellings that come out wrong (e.g. some voices need “ell” rather than “el”, or “zedd”). Keep the map editable from a hidden *Advanced* section in settings so the user can tweak it without code changes. **Z must be “zed” for en-GB/en-AU/en-IE voices and “zee” for en-US.**

6.3 Speaker — timed utterance queue

- Create **one** `SpeechSynthesisUtterance` **per token**. Do not concatenate tokens into one string — punctuation-based pausing is unreliable across engines.
- After each utterance's `onend`, wait `gapMs` (or the token's multiplier × `gapMs`) with `setTimeout`, then speak the next token. Do not pre-queue everything into `speechSynthesis`; drive it token by token so the gap is exact and cancelling is instant.
- Guard against the Chrome bug where `onend` never fires: apply a fallback timeout of `max(1500, 400 × token.text.length)` ms.
- Call `speechSynthesis.cancel()` before starting any item and whenever the user presses Escape, changes settings, or submits early.
- Set `utterance.voice`, `lang`, `rate` (from settings), `pitch = 1`.
- Chrome requires a user gesture before the first utterance. The initial “Press Space to start” interaction satisfies this; also show a warning banner if `speechSynthesis` is undefined.
- Record `performance.now()` at dictation start so keystroke offsets can be measured (used by adaptive speed and the results view).

6.4 Voices — enumeration and picker

- Call `speechSynthesis.getVoices()` on mount *and* on the `voiceschanged` event — in Chrome the list is empty on first call.
- Group voices by language tag (en-GB, en-US, en-AU, en-IE, en-IN, en-ZA, other English, non-English hidden by default). Show name, whether it is a network (“Google ...”) or local voice, and a ► test button that speaks a sample spelling.
- Persist `voiceURI`; if the stored voice is missing on next load, fall back to the first voice matching the stored locale, then to any English voice, and show a toast.
- Offer a **“Random accent each item”** toggle that picks a different English voice per item — closest to real exam variety.

7. Speed control and adaptive timing

7.1 Manual control

- gapMs slider: 200 – 2000 ms, step 50, default 800. Displayed in seconds with one decimal (0.8 s).
- Keyboard: [and] decrease / increase the gap by 50 ms at any time, including mid-item (takes effect from the next token).
- Speech rate slider: 0.7 – 1.3, default 1.0 — this changes how fast each token itself is spoken, independent of the gap.

7.2 Adaptive mode (default on)

After each attempt, adjust gapMs for the next item:

Outcome	Adjustment	Notes
Fully correct, no replay, all keystrokes landed within the gap window	–100 ms	“Within the window” = each keystroke occurred before the next token started speaking.
Fully correct but ≥ 1 keystroke lagged behind the next token	–0 ms (hold)	User got it right but was catching up; don't speed up yet.
1 character wrong or missing	+50 ms	
2+ characters wrong, or a replay used	+150 ms	
3 consecutive fully-correct items	extra –50 ms	Streak bonus to find the edge faster.

- Clamp to the 200–2000 range. Persist the adapted value so the next session starts where the last one ended.
- Show the current gap prominently in the stats bar and animate the number when it changes so the user sees the adaptation happening.
- A “Reset speed to default” button in settings.

8. Training loop and keyboard flow

The entire loop must be operable from the keyboard. There are three visual states on a single page: **Ready**, **Dictating**, **Result**. The settings panel is a side drawer that can be open in any state.

State	What is shown	Keys
Ready	Large empty input with focus; hint text “Press Space or Enter to start”; stats bar	Space / Enter → start item; S → open settings
Dictating	Input receives keystrokes; below it a row of grey token boxes, the currently spoken token highlighted; timer bar for the gap	Type freely; Enter → submit early; R → replay (if allowed, replays from the start after cancel); Esc → abort item (not scored); [] → adjust gap
Result	Diff line (see §9), verdict, adaptation message (“gap 0.8 s → 0.7 s”), next hint	Enter / Space → next item; Backspace → retry the same item (same seed); S → settings

8.1 Input behaviour

- A single controlled <input>, monospace, ~2.5rem font, letter-spacing 0.15em, auto-focused and re-focused on every state change and on any click on the page.
- Uppercase the value on display; keep raw value for scoring (case-insensitive anyway).
- Disable browser autocomplete, autocorrect, spellcheck, and autocapitalize attributes.

- Auto-submit when the typed length equals the target length **and** dictation has finished — or after a configurable silence of 1.5 s once dictation has finished. Enter always submits immediately.
- Never block or debounce keystrokes; state updates must be synchronous with typing. No heavy re-renders in the input's parent while dictating (memoise the token row).

8.2 Live feedback while dictating (optional, toggle in settings)

Below the input, show one box per token. The box for the token currently being spoken is highlighted; boxes the user has already typed correctly turn green immediately, wrong ones turn red. This gives instant feedback on whether the user is keeping up. Default **off** (closer to the exam), available as a learning aid.

9. Scoring and results

9.1 Character-level diff

- Compare typed vs target, case-insensitive. If ignoreSpaces is on, strip spaces from both before comparing (postcodes).
- Use a simple Levenshtein alignment to classify each position as **correct**, **wrong**, **missing** or **extra**. Render the target with per-character colouring and the typed string underneath aligned to it.
- $\text{charAccuracy} = \text{correct} / \text{target length}$. correct = exact match after normalisation.

9.2 Result view content

- Verdict line: ✓ Correct / ✗ 2 errors.
- The diff (two aligned monospace rows).
- The spoken form as text, e.g. “ess · double you · one · ay · oh · double ay” — so the user can see where “double” tripped them.
- Keystroke timing sparkline: one dot per keystroke on a timeline with vertical lines where each token began. Dots after their token's line are shown in amber (lagging).
- Adaptation message and the next gap.

9.3 Session statistics (stats bar + session summary on demand, key T)

- Items attempted, items correct, accuracy %, current streak, best streak.
- Current gap and the gap trajectory over the session (small line chart).
- Error heat-map: which characters or spoken tokens are most often wrong (e.g. “double” tokens, M/N, B/P).
- Per content type accuracy.
- Everything stored in localStorage; a “Clear history” button; an “Export JSON” button.

10. Settings panel (side drawer, key S)

Group	Control	Type / range	Default
Content	Content type	surname / UK postcode / US ZIP / alnum / mixed (checkboxes for mixed)	mixed
Content	Difficulty preset	easy / medium / hard	medium

Group	Control	Type / range	Default
Content	Surname flavour	British / American / both	both
Content	Read whole word first (surnames)	toggle	on
Content	Announce type ("postcode:")	toggle	on
Speech	Voice	picker grouped by locale, with test button	first en-GB
Speech	Random accent each item	toggle	off
Speech	Speech rate	0.7–1.3, step 0.05	1.0
Speech	Zero style	oh / zero / random	random
Speech	Grouping (double/triple)	never / always / random	random
Timing	Gap between characters	0.2–2.0 s, step 0.05	0.8
Timing	Adaptive speed	toggle	on
Timing	Auto-submit silence	0.5–3 s	1.5
Feedback	Live token boxes while dictating	toggle	off
Feedback	Allow one replay (R)	toggle	on
Scoring	Ignore spaces	toggle	on
Advanced	Pronunciation map editor	editable table	as §6.2
Advanced	Reset speed / Clear history / Export JSON	buttons	—

11. UI layout (deliberately minimal)

```

+-----+
| IELTS Spelling Trainer      acc 87% | streak 4 | gap 0.7s | [S] |
+-----+
|
|      postcode
|      _____
|      | S W 1 A   0 A |
|      |-----|
|
| [ess] [double you] [one] [ay]   [oh] [double ay]
|                   ^ current
|
| Enter submit · R replay · Esc abort · [ ] speed
|
+-----+

```

- Single centred column, max-width ~720 px, generous whitespace, system font for chrome, monospace for the input and diff.
- Light theme by default; respect prefers-color-scheme via Tailwind dark: variants — nothing more.
- No animations except the gap timer bar and the gap-number change. No sound effects other than speech (an optional short “ding” on correct is a nice-to-have, default off).
- Settings drawer slides in from the right, 360 px wide, closes with Esc.

12. Iteration plan for Claude Code

Build in the order below and stop after each iteration so the user can test in Chrome. Each iteration must leave the app runnable (`npm run dev`).

#	Deliverable	Done when
1	Scaffold: Next.js + TS + Tailwind, types.ts, seeded RNG, all four generators with Vitest tests, tokeniser with tests, pronunciation map	<code>npm test</code> passes; a debug page prints 20 generated items per type/preset with their token lists.
2	Speech: voices.ts, speaker.ts, VoicePicker; a debug button that speaks any typed string letter by letter with the current gap	User confirms each letter is intelligible in at least one en-GB and one en-US Chrome voice; adjust the pronunciation map accordingly.
3	Training loop: Ready → Dictating → Result on the main page, keyboard flow, diff, manual gap control, localStorage settings	A full practice session works without touching the mouse.
4	Adaptive timing, keystroke timing capture, results sparkline, session stats, streaks, error heat-map	Gap visibly converges over ~15 items; stats survive reload.
5	Settings drawer with every control from §10, random-accent mode, live token boxes, replay, retry-same-seed, export JSON	Every setting in §10 is wired and persisted.
6	Polish and robustness: Chrome onend fallback, voice-missing fallback, focus management, dark mode, README	No console errors across a 50-item session; README documents keys and settings.

Expected effort: one evening of collaborative work with Claude Code for iterations 1–4, a second short session for 5–6. Prioritise correctness of timing and speech over anything visual.

13. Known risks and mitigations

Risk	Mitigation
Chrome <code>speechSynthesis</code> stops firing <code>onend</code> (long-standing bug)	Fallback timeout per utterance (§6.3); call <code>cancel()</code> before each item; keep utterances short.
Voice list empty on first load	Listen to <code>voiceschanged</code> ; retry <code>getVoices()</code> up to 10× at 100 ms.
Letters mispronounced by some voices	Spelled-out map (§6.2) + user-editable overrides.
Speech blocked until user gesture	Start requires a key press; show a banner if <code>speechSynthesis</code> is unavailable.
Firefox/Safari voice quality is poor or voices differ	Document Chrome as the target; do not try to normalise across engines in v1.
Generated surnames occasionally look silly	Acceptable for training; the skill is keystroke tracking, not realism. Post-filters in §5.1 remove the worst cases.
Typing lag from React re-renders	Keep the input in its own memoised component; store keystroke timings in a ref, not state.
localStorage unavailable (private mode)	Wrap all access in try/catch; run in-memory if it fails.

14. Future extensions (not for v1)

- API route that fetches real surname distributions (e.g. public census tables) and sorts them by spelling difficulty.

- Full IELTS Section 1 form-filling simulation (name, address, phone, dates) with a mock answer sheet.
- Background noise / echo overlay to simulate poor audio.
- Spaced-repetition of the user's personally weak tokens (from the error heat-map).
- Export of session history to CSV for long-term tracking.

Suggested opening prompt for Claude Code:

“Read the attached specification for the IELTS Listening Spelling Trainer. Implement iteration 1 from Section 12 exactly as described — Next.js App Router with TypeScript and Tailwind, seeded RNG, the four generators and the tokeniser with Vitest tests, and the pronunciation map — then stop and show me how to run the debug page. Do not bundle any lists of names or postcodes; everything must be generated. Ask me only if a decision marked [OPEN] blocks you.”